Informe técnico y README

Ejercicios de Estructuras de Datos y Algoritmos

Autor	Maycol Leon (msleon18)
Repositorio	https://github.com/msleon18/proyecto-final-eda
Organización	Un solo repositorio, una rama por ejercicio
Fecha	Octubre de 2026

1. Descripción general

Este repositorio reúne los ejercicios del curso. Siguiendo la indicación del profesor, todo está en un **único repositorio** y **cada ejercicio vive en su propia rama**, con su código y su historial de commits. La rama **main** contiene el proyecto final, el README con el índice de ejercicios y este informe (carpeta `docs/`). Cada ejercicio implementa una estructura de datos desde cero, sin usar las colecciones ya hechas del lenguaje para la estructura central.

Ejercicio	Rama	Estructura / tema	Tecnologías	Despliegue
Reto 3 - Listas	reto-03	Lista enlazada simple y lista doblemente enlazada (canciones e historial de navegación)	JavaScript, React 19, Vite, React Router 7	No (local)
Reto 4 - Pila de libros	pilas-libros	Pila (Stack, LIFO)	JavaScript, React 19, Vite	No (local)
Reto 5 - Cola del cajero	cola-cajero	Cola (Queue, FIFO)	JavaScript, React 19, Vite	No (local)
Reto 8 - Árbol binario	challenge-08	Árbol binario de búsqueda y recorridos	JavaScript, React 19, Vite, react-d3-tree	No (local)
Reto 9 - Árbol n-ario	challenge-09	Árbol n-ario (menú lateral dinámico)	JavaScript, React 19, Vite, lucide-react	No (local)
Reto 10 - Grafos	challenge-10	Grafo no dirigido con lista de adyacencia (ciudades y personas)	JavaScript, React 19, Vite, react-force-graph-2d	No (local)
Reto 11 - Trie + Heap	challenge-11	Trie y Min Heap: buscador Top K por prefijo	Java (JDK 17+), JUnit 5	No (consola)
Proyecto final	main / proyecto-final	Pila, Cola, BST, Min-Heap y Grafo (BFS/DFS) con persistencia	React + Vite, Node.js + Express, PostgreSQL	**Sí** (Netlify + Render)

2. Herramientas utilizadas

Herramienta	Para qué se usó
Git	Control de versiones: un commit por cada pieza terminada, con mensajes claros (feat, fix, docs, test, chore, style) y una rama por ejercicio.
GitHub	Repositorio remoto público. Contiene las ramas de cada ejercicio y el README con el índice con enlaces a cada rama.
Visual Studio Code	Editor de código, con Extension Pack for Java y la terminal integrada (PowerShell).
JDK (Temurin 25) - javac / java	Compilar y ejecutar el Reto 11 en Java. El código requiere Java 17 o superior (usa records).

Herramienta	Para qué se usó
JUnit 5 (Console Standalone 1.11.4)	Pruebas automáticas del Reto 11, ejecutadas desde la terminal sin Maven ni Gradle.
Node.js y npm	Ejecutar los proyectos de React y el backend del proyecto final; instalar dependencias.
Vite	Servidor de desarrollo y empaquetado (build) de las aplicaciones React.
React 19	Interfaz de usuario con componentes funcionales y hooks (useState). Visualización de cada estructura.
Express 5, cors, dotenv, pg	Backend del proyecto final: API REST, manejo de CORS, variables de entorno y conexión a PostgreSQL.
PostgreSQL (Supabase)	Base de datos del proyecto final: historial de operaciones y transacciones.
Render y Netlify	Despliegue del backend (Render) y del frontend (Netlify) del proyecto final.

3. Especificaciones técnicas

3.1 Reto 11 - Buscador inteligente (Trie + Heap, Java)

Dado un prefijo, devuelve los K productos más populares que empiezan con él. El Trie encuentra rápido todos los productos con ese prefijo, y un Min Heap de tamaño K conserva solo los mejores sin ordenar toda la lista: su raíz es siempre el menos popular de los K guardados, así que basta compararla con cada candidato nuevo.

Clase	Responsabilidad
Product (record)	Dato básico: nombre y popularidad.
TrieNode	Nodo del árbol: mapa de hijos (HashMap de carácter a nodo), bandera isEndOfWord y el producto guardado.
Trie	insert, search y searchTopK. Normaliza a minúsculas y recorre el subárbol en profundidad (DFS).
MinHeap<T>	Heap genérico implementado a mano sobre una lista, con Comparator: push, pop, peek, heapify, percolateUp y percolateDown.
ProductLoader	Lee un CSV (name,popularity), omite el encabezado y avisa de las líneas inválidas sin detener el programa.
App / Main	Menú interactivo por consola (buscar por prefijo, cambiar K, agregar producto, búsqueda exacta, simulación letra por letra) y pruebas de ejemplo.

- **Complejidad:** insert y search O(L), con L la longitud de la palabra; push y pop O(log n); peek O(1); heapify O(n); searchTopK O(M log K), con M los productos bajo el prefijo.
- **Datos:** 320 productos en `data/products.csv`.
- **Pruebas:** 28 pruebas JUnit (MinHeapTest 10, TrieTest 15, ProductLoaderTest 3), incluida una que compara el resultado con una búsqueda por fuerza bruta sobre 1000 productos aleatorios.

3.2 Proyecto final - Visualizador interactivo de estructuras de datos

Plataforma web para visualizar y manipular cinco estructuras: Pila, Cola, Árbol Binario de Búsqueda, Min-Heap y Grafo con recorridos BFS y DFS. Cada operación del usuario viaja a un backend que ejecuta la lógica real de la estructura y guarda un registro en la base de datos. Flujo: **Frontend (React) > API REST (axios) > Backend (Express) > PostgreSQL**.

Capa	Tecnología	Detalle
Frontend	React 19, Vite, axios	Un componente visual por estructura (components/) y un servicio por estructura (services/) que consume la API.

Capa	Tecnología	Detalle
Backend	Node.js, Express 5, cors, dotenv	Rutas /api/stack, /api/queue, /api/bst, /api/heap, /api/grafo y /api/health. La lógica de cada estructura está en structures/.
Base de datos	PostgreSQL (Supabase), pg	Tabla operaciones: estructura, tipo_operacion, valor, estado_resultante (estado en JSON) y creado_en.
Transacciones	BEGIN / COMMIT / ROLLBACK	El reinicio de una estructura borra su historial e inserta un registro de reset en un solo bloque atómico; si algo falla, se revierte todo.
Configuración	Variables de entorno	DATABASE_URL (backend, archivo .env fuera de Git) y VITE_API_URL (frontend, ejemplo en .env.example).

3.3 Retos de React (3, 4, 5, 8, 9 y 10)

Todos son aplicaciones React 19 creadas con Vite. La estructura de datos está implementada como una clase de JavaScript propia, separada de la interfaz, y el estado de la pantalla se maneja con hooks.

Rama	Estructura (archivos clave)	Funcionalidad
reto-03	utils/LinkedList.js y utils/DoublyLinkedList.js	Página de canciones (lista enlazada simple) y página de historial de navegación (lista doble, avanzar y retroceder), con React Router y navegación por botones.
pilas-libros	models/Stack.js	Push y pop de libros (nombre, ISBN, autor, editorial); marca el tope de la pila; validaciones de campos vacíos.
cola-cajero	models/Queue.js	Enqueue y dequeue de clientes (nombre y monto a retirar, con hora de llegada); marca quién está en ventanilla.
challenge-08	App.jsx (clases Node y BinarySearchTree)	Inserción, búsqueda y recorridos PreOrder, InOrder y PostOrder; árbol dibujado con react-d3-tree.
challenge-09	menuTree.js (MenuNode) y components/Sidebar.jsx	Árbol n-ario como menú lateral con submenús; componente recursivo; panel con las propiedades del nodo elegido.
challenge-10	graph.js (clase SocialGraph) y components/GraphComponent.jsx	Grafo no dirigido con lista de adyacencia: ciudades y personas como vértices; filtro de habitantes por ciudad; visualización con física y zoom.

4. Despliegues

Solo el **proyecto final** se desplegó. Los demás ejercicios se ejecutan en local.

Componente	Plataforma	Dirección
Frontend	Netlify	https://iridescent-treacle-8d3e02.netlify.app
Backend	Render (plan gratuito)	https://proyecto-final-eda.onrender.com
Base de datos	Supabase (PostgreSQL)	Servicio administrado, conectado al backend con la variable DATABASE_URL

- En el plan gratuito de Render el backend se duerme tras un periodo sin uso, por lo que la primera petición puede tardar hasta 50 segundos.
- El estado de cada estructura vive en la memoria del servidor: si Render lo reinicia, las estructuras vuelven a estar vacías, aunque el historial de operaciones en la base de datos se conserva.

5. Cómo ejecutar

Retos de React (reto-03, pilas-libros, cola-cajero, challenge-08, 09 y 10): clonar la rama del ejercicio, instalar y arrancar. Se abre en `http://localhost:5173`.

```
git clone -b challenge-08 https://github.com/msleon18/proyecto-final-eda.git
cd proyecto-final-eda
npm install
npm run dev
```

Reto 11 (rama challenge-11, requiere JDK 17 o superior):

```
javac -d bin src/*.java
java -cp bin App        # menú interactivo

# Pruebas (PowerShell): JUnit descargado en lib/
$JUNIT = "lib/junit-platform-console-standalone-1.11.4.jar"
javac -d bin -cp $JUNIT src/*.java test/*.java
java -jar $JUNIT execute --class-path bin --scan-class-path
```

Proyecto final (rama main): requiere una base PostgreSQL y crear `backend/.env` con DATABASE_URL.

```
# Backend
cd backend
npm install
npm run dev

# Frontend (copiar frontend/.env.example como frontend/.env y ajustar VITE_API_URL)
cd frontend
npm install
npm run dev
```

6. Organización del repositorio y control de versiones

- **Una rama por ejercicio** (9 ramas en total), cada una con su código y su historial propio. Las ramas no se mezclan entre sí.
- **main** es la rama principal: proyecto final, README con el índice de ejercicios y este informe en `docs/`.
- **.gitignore** excluye lo que no debe subirse: node_modules, archivos compilados (bin, .class), librerías descargadas (lib) y archivos .env.
- **Secretos fuera de Git:** las claves de conexión van en archivos .env que no se versionan; el repositorio incluye solo un .env.example como guía.
- **Commits pequeños y descriptivos**, uno por cada pieza terminada (por ejemplo, feat: add Trie, test: add JUnit tests).